

```

#include <stdio.h>
#include <stdlib.h>
struct Node{
    int data;
    struct Node* next;
};
struct Node* head=NULL;
struct Node* head1=NULL;

void insertatBeg(int val)
{
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=val;
    newNode->next=head;
    head=newNode;
}
void insertatBeg1(int val)
{
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=val;
    newNode->next=head1;
    head1=newNode;
}

void insertatEnd(int val)
{
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=val;
    newNode->next=NULL;
    if(head==NULL)
    {
        head=newNode;
    }
    else{
        struct Node* t=head;
        while(t->next!=NULL)
        {
            t=t->next;
        }
        t->next=newNode;
    }
}
void insertatEnd1(int val)
{
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=val;
    newNode->next=NULL;
    if(head1==NULL)
    {
        head1=newNode;
    }
}

```

```

    }
    else{
        struct Node* t=head1;
        while(t->next!=NULL)
        {
            t=t->next;
        }
        t->next=newNode;
    }
}
}
void insertatMid(int val,int pos)
{
    if(pos==1)
    {
        insertatBeg(val);
    }
    else{
        struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
        newNode->data=val;
        struct Node* t=head;
        for(int i=1;i<pos-1 && t!=NULL;i++)
        {
            t=t->next;
        }
        if (t == NULL) {
            printf("Position out of Bounds\n");
            free(newNode);
            return;
        }
        newNode->next=t->next;
        t->next=newNode;
    }
}
void insertatMid1(int val,int pos)
{
    if(pos==1)
    {
        insertatBeg1(val);
    }
    else{
        struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
        newNode->data=val;
        struct Node* t=head1;
        for(int i=1;i<pos-1 && t!=NULL;i++)
        {
            t=t->next;
        }
        if (t == NULL) {
            printf("Position out of Bounds\n");
            free(newNode);

```

```

        return;
    }
    newNode->next=t->next;
    t->next=newNode;
}

void deletefromBeg()
{
    if(head==NULL)
    {
        printf("Empty list");
        return;
    }
    struct Node* t=head;
    head=head->next;
    free(t);
    printf(" Element deleted Succesfully");
}

void deletefromEnd()
{
    if(head==NULL)
    {
        printf("Empty list");
        return;
    }
    if(head->next==NULL)
    {
        free(head);
        head=NULL;
        return;
    }
    struct Node* t=head;
    while(t->next->next!=NULL)
    {
        t=t->next;
    }
    free(t->next);
    t->next=NULL;
}

void deletefromMid(int pos)
{
    if(head==NULL)
    {
        printf("Empty list");
        return;
    }
    if(pos==1)
    {

```

```

        deletefromBeg();
        return;
    }
    struct Node* t=head;
    for(int i=1;i<pos-1 && t!=NULL;i++)
    {
        t=t->next;
    }
    if (t == NULL || t->next == NULL)
    {
        printf("Invalid position\n");
        return;
    }
    struct Node* del=t->next;
    t->next=del->next;
    free(del);
    printf(" Element at %d deleted Succesfully",pos);
}

void searchelement(int key)
{
    int pos=1;
    if(head==NULL)
    {
        printf(" Empty list");
        return;
    }
    struct Node* t=head;
    while(t!=NULL)
    {
        if(t->data==key)
        {
            printf(" Element found at position : %d \n",pos);
            return;
        }
        t=t->next;
        pos++;
    }
    printf(" Element not found \n");
}

void display()
{
    if(head==NULL)
    {
        printf(" Empty list \n");
        return;
    }
    struct Node* t=head;
    while(t!=NULL)
    {

```

```

        printf("%d->",t->data);
        t=t->next;
    }
    printf("null");
}
void display1()
{
    if(head1==NULL)
    {
        printf(" Empty list \n");
        return;
    }
    struct Node* t=head1;
    while(t!=NULL)
    {
        printf("%d->",t->data);
        t=t->next;
    }
    printf("null");

}
void merge()
{
    if(head==NULL)
    {
        printf(" Empty list \n");
        return;
    }
    struct Node* t=head;
    while(t->next!=NULL)
    {
        t=t->next;
    }
    struct Node* temp=head1;
    if(t->next==NULL)
    {
        t->next=temp;
    }
    struct Node* tp=head;
    while(tp!=NULL)
    {
        printf("%d->",tp->data);
        tp=tp->next;
    }
    printf("null");
}

int main(){
    int choice,pos,val,key;

```

```

while(1){
    printf("\n1.Insert at beginning \n2.Insert at End \n3.Insert at Middle
\n4.Delete at Beginning \n5.Delete at End \n6.Delete at Middle \n7.Search
\n8.Display \n9.Sort \n10.Exit  \n11.secondlistinsertatbeg
\n12.secondlistinsertatend \n13.secondlistinsertatmid \n14.displaysecondlist
\n.15.merge");
    printf("\n Enter choice:");
    scanf("%d",&choice);
    switch(choice)
    {
        case 1:
            printf("enter element to insert at the beginning:");
            scanf("%d",&val);
            insertatBeg(val);
            break;

        case 2:
            printf("enter element to insert at end:");
            scanf("%d",&val);
            insertatEnd(val);
            break;

        case 3:
            printf(" Enter element to insert : ");
            scanf("%d",&val);
            printf(" Enter the Position : ");
            scanf("%d",&pos);
            insertatMid(val,pos);
            break;

        case 4:
            printf(" Delete from Beginning");
            deletefromBeg();
            break;

        case 5:
            printf(" Delete from End");
            deletefromEnd();
            break;

        case 6:
            printf(" Enter the position to delete : ");
            scanf("%d",&pos);
            deletefromMid(pos);
            break;

        case 7:
            printf(" Enter key value to Search : ");
            scanf("%d",&key);
            searchelement(key);
    }
}

```

```

break;

case 8:
printf("\nThe list elements are : ");
display();
break;

case 11:
printf("enter element to insert at the beginning:");
scanf("%d",&val);
insertatBeg1(val);
break;

case 12:
printf("enter element to insert at end:");
scanf("%d",&val);
insertatEnd1(val);
break;

case 13:
printf(" Enter element to insert : ");
scanf("%d",&val);
printf(" Enter the Position : ");
scanf("%d",&pos);
insertatMid1(val,pos);
break;

case 14:
printf("\nThe  second list elements are : ");
display1();
break;

case 15:
printf("the merged list is:");
merge();
break;

case 10:
return 0;
break;

default:
printf(" \n Invalid choice \n");

```

```

}

```

```

}

```

```

}

```